

Chapter 7

The μ ML Process Model, v2.0.

Contents

7.	The μ ML Process Model.....	2
7.1	Elements of the Process Model	2
7.2	Notation for the Process Model	3
7.3	Rules of the Process Model	4
7.4	Process Model Examples	5
7.5	Process Metamodel	6
7.6	Model Transformations	10
7.7	Differences from UML	10
7.8	References	11

7. The μ ML Process Model

The μ ML Process Model is a set of dataflow graphs, which may be created by hand during the design phase or may be created automatically by model transformation. When sketched by hand, a Process Model is drawn as a set of process diagrams containing atomic actions linked by dataflow transitions. When created by model transformation, a Process Model is obtained from a μ ML Impact Model by mapping each task to a process, refining its impacts to atomic actions and dataflows.

The purpose of the Process Model is to represent the data processing behaviour of a system. Actions in the model represent atomic computational steps, such as reading, writing, fetching or storing data. The Process Model defines how each business task breaks down into computational steps. The types on the dataflows correspond to objects in the μ ML Object Model. The Process Model complements the μ ML State Model by defining the processing behaviour of some of its transitions.

7.1 Elements of the Process Model

The Process Model consists of the following concepts:

- **Process** – is a process of the system, a refinement of one atomic business task. A process typically consists of a set of concrete actions. A general process will consist of a choice of alternative specific processes.
- **Action** – is an atomic action, or one computational step performed by a process. There are many different kinds of action.
- **InputAction** – is a kind of action denoting fetching user input (from a GUI).
- **OutputAction** – is a kind of action denoting sending display output (to a GUI).
- **FetchAction** – is a kind of action denoting fetching an object from a datastore.
- **StoreAction** – is a kind of action denoting storing an object in a datastore. This includes both the initial insertion and the replacement of the stored object.
- **CreateAction** – is a kind of action denoting the creation of an object in memory.
- **UpdateAction** – is a kind of action denoting the modification of an object in memory.
- **DeleteAction** – is a kind of action denoting the deletion, or archival of an object. A deleted object is removed from the datastore.

The Process Model also consists of the following connectives:

- **Dataflow** – denotes a flow of data connecting a source action to a target action. The dataflow is labelled with a datum, denoting the kind of data produced by the source action and consumed by the target action.

The Process Model also consists of the following features:

- **Datum** – is a kind of feature denoting the label on a dataflow transition. There are three kinds of datum, denoting different kinds of data values.
- **Object** – is a kind of datum denoting a whole encapsulated object (in memory).
- **Content** – is a kind of datum denoting the assignable values of an object, excluding any references to other objects.
- **Identity** – is a kind of datum denoting just the identifying values of an object.

The Process Model also has the following regions:

- **Diagram** – is a kind of region containing a set of Processes. The diagram is the root container for a Process Model.

7.2 Notation for the Process Model

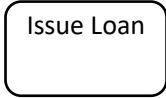
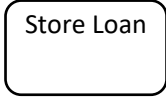
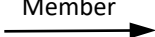
Process: denotes a process performed by the system, which breaks down into a set of actions. The process box is drawn large, to contain all of its actions.	
Action: denotes one computational step performed as part of a Process. The actions are drawn small, to fit inside their owning process box.	
Dataflow: denotes a flow transferring data from the source to the target action. The arrow flows from the source to the target action and is labelled with the kind of data transferred.	

Figure 7.1: Notation for the Process Model

Figure 7.1 illustrates the notation used for concepts and connectives in the μ ML Process Model. It adopts standard μ ML conventions in the concepts, connectives and features used. The icon for a Process is drawn as a rectangle with rounded corners. This distinguishes it from the μ ML state icon, which is drawn as a flat oval with straight upper and lower edges and rounded sides (see chapter 6). The icon for a dataflow is a labelled arrow with a filled arrowhead. This distinguishes it from a state transition, which has an open arrowhead (see chapter 6).

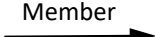
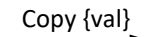
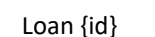
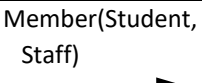
Object Dataflow: is labelled with the name of the object, and denotes a flow transferring the whole object.	
Content Dataflow: is qualified with {val} after the object name, and denotes a flow transferring the set of assignable attribute values of the object.	
Identity Dataflow: is qualified with {id} after the object name, and denotes a flow transferring the set of identifying values of the object.	
General Object Dataflow: is labelled with the name of the general object, followed by all of its concrete subtypes in parentheses. (The {id} and {val} annotations may follow).	

Figure 7.2: Different features of a dataflow

Figure 7.2 illustrates the different styles in which datum labels may be added to a dataflow. A dataflow must always have a label. If the dataflow has a simple object name, it denotes a flow transferring the whole encapsulated object, such as when the object is fetched or stored. If the object name is qualified by {val}, this denotes a flow transferring all the assignable attribute values of the object, excluding any references, such as when the object is first initialised, or updated. If the object name is qualified by {id}, this denotes a flow transferring only the identifying values of the object, such as when the object is about to be looked up in a datastore.

If the object is a general object, with subtypes, then the flow is always labelled with the most general kind of object participating in the process, followed by a list of its concrete subtypes in parentheses. This list may not contain any intermediate generalisations.

7.3 Rules of the Process Model

The Process Model has the following syntactic and semantic rules:

- **Main impact** – every process can identify its main impact in: *[create, update, delete, read]* from the related task in the Impact Model. This will be the first impact in the above list to appear in the task's set of impacts (a consistency rule).
- **Main object** – every process can identify its main object from the target of its main impact in the corresponding Impact Model. This object is the focus of the process (a consistency rule).
- **General process** – a general process is one whose main object is not derived from any other object. The main object may either stand alone, or must not be the source of any generalisation in the related Impact Model (a definition rule).
- **Special process** – a special process is one whose main object is derived from some other object. The main object is the source of a generalisation in the related Impact Model (a definition rule).
- **Process filtering** – the Process Model consists only of general processes, to avoid process duplication. Special processes are assumed to be trivial variants of general processes and are not separately modelled (a definition rule).
- **Process refinement** – every general process is broken down into a set of atomic actions, connected by dataflows, which carry the kind(s) of data transferred (a definition rule).
- **Simple dataflow** – a simple dataflow is one carrying the label of a single object, where this object stands alone in the corresponding Impact Model (a definition rule).
- **Compound dataflow** – a compound dataflow is one carrying the label of a general object and all of its concrete subtype objects, which are its refinements in the corresponding Impact Model (a definition rule).
- **Identity dataflow** – an identity dataflow is a simple or compound dataflow whose label is followed by {id} and denotes the flow of just the identifying properties of the object(s) (a definition rule).
- **Content dataflow** – a content dataflow is a simple or compound dataflow whose label is followed by {val} and denotes the flow of all the initialisation values of the object(s), excluding any references (a definition rule).
- **Datum production** – if a dataflow runs from a source action, this action must provide the datum named on the dataflow (a semantic rule).
- **Datum consumption** – if a dataflow runs to a target action, this action must require the datum named on the dataflow (a semantic rule).
- **Multiple inflows** – if an action is the target of multiple dataflows, then it must require all of the data on the incoming flows (a semantic rule).
- **Multiple outflows** – if an action is the source of multiple dataflows, then it must provide all of the data on the outgoing flows (a semantic rule).
- **Datum distribution** – if multiple outflows carry the same datum label, this means that the source action distributes the datum to multiple downstream actions (a semantic rule).
- **Action dependency** – if a dataflow runs from a source action to a target action, this means that the source action must execute before the target action (a semantic rule).
- **Partial order** – the action dependencies in a process define a partial order. The upstream actions execute before the downstream actions, but within this there may be some flexibility in order of execution (a semantic rule).
- **Implicit boundary** – the diagram is the implicit boundary of the Process Model. Every contained process is part of the model (a completeness rule).

7.4 Process Model Examples

Some examples are given below of Processes taken from the Process Model of a Lending Library. The whole Process Model is too large to display inside one frame, so each of the figures below displays a selection of different Processes.

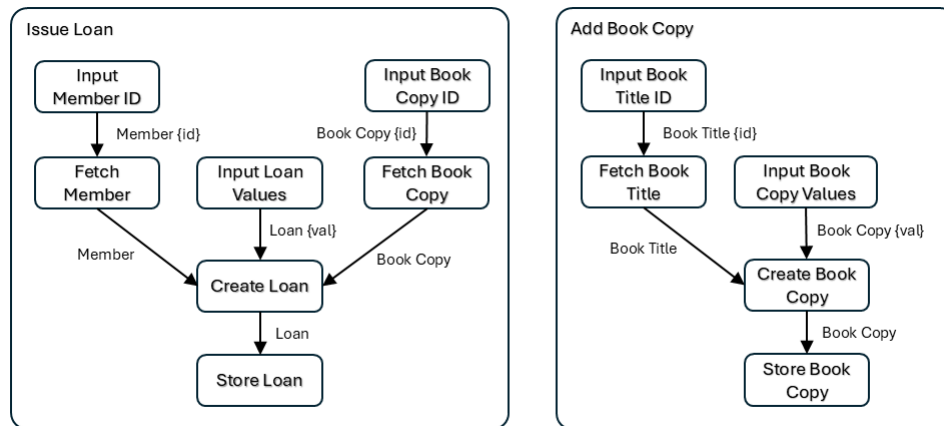


Figure 7.3: Process Model for *Issue Loan* and *Add Book Copy*

Figure 7.3 illustrates Process Model diagrams for two processes carried out in a Lending Library that create new information. The first of these processes, *Issue Loan*, is refined into the following seven computational actions, each of which is a simple algorithmic step:

- **Input Member ID** – is an *InputAction* requesting the identity of the *Member* from the GUI.
- **Input Book Copy ID** – is an *InputAction* requesting the identity of the *Book Copy* from the GUI.
- **Input Loan Values** – is an *InputAction* requesting the attribute values of the *Loan* to be issued, such as the issue date and the due date.
- **Fetch Member** – is a *FetchAction* that retrieves the *Member* object from the datastore, using the *Member {id}* as a search key.
- **Fetch Book Copy** – is a *FetchAction* that retrieves the *Book Copy* object from the datastore, using the *Book Copy {id}* as a search key.
- **Create Loan** – is a *CreateAction* that constructs the *Loan* object from the supplied *Member* and *Book Copy* objects and the other *Loan {val}* attribute values for the issue date and due date.
- **Store Loan** – is a *StoreAction* that stores the resulting *Loan* object in the datastore.

These actions are linked by Dataflows expressing the kind of data that flows between the actions. These flows are of three kinds:

- *Object Dataflow* – for example, *Member*, *Book Copy* and *Loan* are passed as encapsulated objects between some actions.
- *Identity Dataflow* – for example, the *Member {id}* and *Book Copy {id}* are passed from input actions to the actions that fetch the corresponding objects from the datastore.
- *Content Dataflow* – for example, the *Loan {val}* additional attribute values supplied by an input action in order to create the *Loan*.

The kind of dataflow required is determined by what the following target action needs. For example, any *Fetch Action* will require the ID of the object to fetch; and any *Create Action* will require the references to dependent objects and the values of attributes to be used in a constructor.

Similarly, the process *Add Book Copy* breaks down into a number of computational steps:

- **Input Book Title ID** – is an *InputAction* requesting the identity of the book title from the GUI.

- **Input Book Copy Values** – is an *InputAction* requesting the attribute values of the *Book Copy* to be issued, such as its unique copy ID.
- **Fetch Book Title** – is a *FetchAction* that retrieves the *Book Title* object from the datastore, using the *Book Title {id}* as a search key.
- **Create Book Copy** – is a *CreateAction* that constructs the *Book Copy* object from the supplied *Book Title* object and the other *Book Copy {val}* attribute values for the copy.
- **Store Book Copy** – is a *StoreAction* that stores the resulting *Book Copy* object in the datastore.

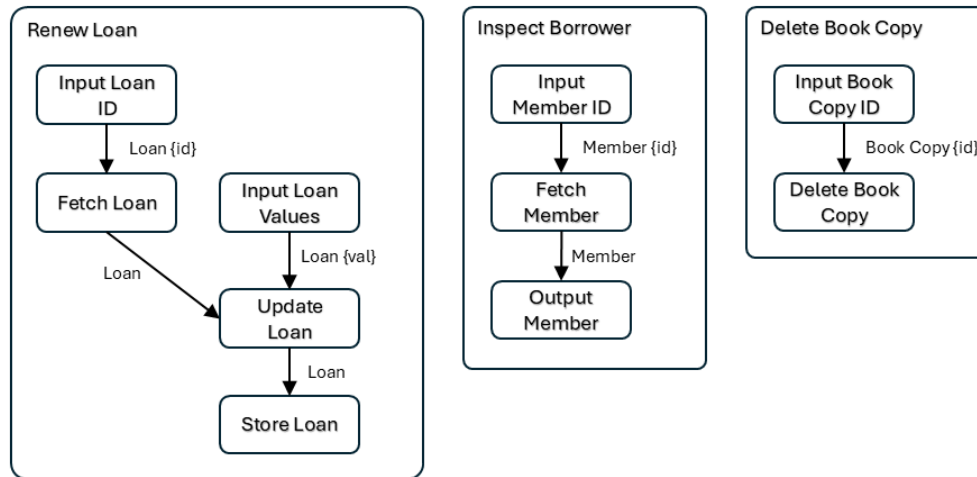


Figure 7.4: Process Model for *Renew Loan*, *Inspect Borrower* and *Delete Book Copy*

Both of these examples in figure 7.3 show how any process that creates new information must break down into actions that retrieve existing information from the datastore, in order to link it with new information supplied at the user interface. Figure 7.4 illustrates three more Process Model diagrams taken from the Lending Library case study. These diagrams focus on processes that update, inspect and delete information. As before, in order to access existing objects for updating, or displaying their contents, they have to be identified and fetched from the datastore. When deleting an object from the datastore, it is sufficient to provide its ID.

Working out how any process breaks down into more primitive actions is determined from the main CRUD purpose of the process. These actions can be determined automatically according to information present in the *µML Impact Model*.

7.5 Process Metamodel

The elements of the *µML* Process Model may be defined in a metamodel. In figure 7.5, we use a metamodel written in the *ReMoDeL* metamodel definition language [1]. This extends and redefines concepts defined in the *ReMoDeL* metamodel of the *µML* Core Model, given in Chapter 2.

```

# The Process Model from muML (micro Modelling Language). This consists of a
# Diagram containing Processes, Actions and Dataflows. Each Process denotes
# one atomic business task, which is refined into algorithmic steps known as
# Actions. The Actions have many specialised subtypes, such as: InputAction,
# OutputAction, CreateAction, UpdateAction, DeleteAction, FetchAction and
# StoreAction. Dataflows connect the different Actions, describing what data
# flows from one Action to the next. Each Datum is a datatype, denoting either
# a whole object, or its value-fields, or its identifier-fields.

metamodel ProcessModel inherit CoreModel {

# Process is a kind of Concept denoting a computational process. A Process
# is owned by a Diagram. A Process contains a list of Actions connected by
# Dataflows.

```

```

concept Process inherit Concept {
  reference owner : Diagram
  attribute kind : String
  component actions : Action[]
  component dataflows : Dataflow[]
}

# Datum is a kind of Feature labelling a piece of data. It is used variously
# to represent just the ID fields, or all value fields, or a whole object.
# The name of the Datum describes a simple object or general object. If this
# is general, then the kinds is a list of names of its concrete subtypes.

concept Datum inherit Feature {
  component kinds : String
  operation isIdentity : Boolean { false }
  operation isContent : Boolean { false }
  operation isObject : Boolean { false }
}

# Identity is a kind of Datum denoting just the ID fields of some object.

concept Identity inherit Datum {
  operation isIdentity : Boolean { true }
}

# Content is a kind of Datum denoting all the field values of some object.

concept Content inherit Datum {
  operation isContent : Boolean { true }
}

# Object is a kind of Datum denoting the entire object and its properties.

concept Object inherit Datum {
  operation isObject : Boolean { true }
}

# Action is a kind of Concept denoting an algorithmic step. An Action is
# owned by a Process. Actions have incoming and outgoing Dataflows, which
# carry an abstract piece of Data. From this, we can determine the inputs
# and outputs of each Action.

concept Action inherit Concept {
  reference owner : Process
  operation incoming : Dataflow[] {
    owner.dataflows.select(flow | flow.target = self)
  }
  operation outgoing : Dataflow[] {
    owner.dataflows.select(flow | flow.source = self)
  }
  operation inputs : Datum[] {
    self.incoming.collect(flow | flow.datum)
  }
  operation outputs : Datum[] {
    self.outgoing.collect(flow | flow.datum)
  }
  operation isInput : Boolean { false }
  operation isOutput : Boolean { false }
  operation isFetch : Boolean { false }
  operation isStore : Boolean { false }
  operation isCreate : Boolean { false }
  operation isUpdate : Boolean { false }
  operation isDelete : Boolean { false }
}

# InputAction is a kind of Action denoting user input from the GUI.
# An InputAction may input just the ID, or all the field values of
# some object.

```

```

    concept InputAction inherit Action {
        attribute id : Boolean
        operation isID : Boolean { id }
        operation isValue : Boolean { not id }
        operation isInput : Boolean { true }
    }

# OutputAction is a kind of Action denoting output display to the GUI.

    concept OutputAction inherit Action {
        operation isOutput : Boolean { true }
    }

# FetchAction is a kind of Action denoting retrieval from a database.

    concept FetchAction inherit Action {
        operation isFetch : Boolean { true }
    }

# StoreAction is a kind of Action denoting storage to a database.
# A StoreAction may insert for the first time, or replace.

    concept StoreAction inherit Action {
        attribute again : Boolean
        operation isInsert : Boolean { not again }
        operation isReplace : Boolean { again }
        operation isStore : Boolean { true }
    }

# CreateAction is a kind of Action denoting object creation.

    concept CreateAction inherit Action {
        operation isCreate : Boolean { true }
    }

# UpdateAction is a kind of Action denoting object field-assignment.

    concept UpdateAction inherit Action {
        operation isUpdate : Boolean { true }
    }

# DeleteAction is a kind of Action denoting object deletion. A
# DeleteAction may delete, or just archive the deleted object.

    concept DeleteAction inherit Action {
        attribute archive : Boolean
        operation isDelete : Boolean { true }
        operation isArchive : Boolean { archive }
    }

# Dataflow is a kind of Connective denoting a dataflow, transferring data
# between one Action and the next. A Dataflow is owned by a Process.

    concept Dataflow inherit Connective {
        reference owner : Process
        reference source : Action
        reference target : Action
        component datum : Datum
    }

# Diagram is the top-level region, enclosing the whole Process Model.

    concept Diagram inherit Region {
        component processes : Process[]
    }
}

```

Figure 7.5: Metamodel of the Process Model in the ReMoDeL language.

This provides the necessary elements of the Process Model via a number of intermediate abstractions in the metamodel hierarchy:

- **Process** – is a kind of core Concept denoting a Process, and is owned by a Diagram. A Process is derived from an atomic business Task and breaks down into more detailed Actions. A Process owns its Actions, and also the Dataflows that connect its Actions.
- **Action** – is a kind of core Concept, denoting an abstract computational step. There are several kinds of Action, denoting different concrete computational steps;
- **InputAction** – is a kind of Action denoting user input from the GUI. This indicates whether the requested input is {id} information only, otherwise it is full {val} information, which means the attribute values of an object, excluding any references;
- **OutputAction** – is a kind of Action denoting output to be displayed at the GUI;
- **FetchAction** – is a kind of Action denoting retrieving an object from a datastore. This will require the object's {id} to retrieve it.
- **StoreAction** – is a kind of Action denoting storing a whole object in a datastore. This indicates whether the object is being stored *again*, viz. replaced in the datastore, otherwise it is being inserted for the first time in the datastore;
- **CreateAction** – is a kind of Action denoting object construction. This will require all the necessary construction arguments, supplied as objects or values;
- **UpdateAction** – is a kind of Action denoting object modification. This will require the target object to modify and a new set of attribute values;
- **DeleteAction** – is a kind of Action denoting object deletion, or archival. This indicates whether *archive* is requested, otherwise the object will be deleted.
- **Dataflow** – is a kind of core Connective denoting the flow of data from one Action to another.
- **Datum** – is a kind of core Feature, labelling some abstract datum. Its name describes the name of an object. If this object is general, its kinds are given as a list of its concrete subtypes.
- **Object** – is a kind of Datum denoting a whole encapsulated object;
- **Content** – is a kind of Datum denoting the initialisation values of an object;
- **Identity** – is a kind of Datum denoting the identifying values of an object;
- **Diagram** – is a kind of core Region denoting the Diagram containing all the Processes.

The Process Model has to resolve a problem of task duplication, resulting from tasks that deal with general (supertype) and specific (subtype) objects. An example of this problem is where the Lending Library has *Enrol Member* and *Retire Member* to add and remove members, but *Member* is abstract, with two concrete subtypes: *Student Member* and *Staff Member*. A complete Impact Model will introduce extra *create* and *delete* Tasks for each concrete object.

When transforming this to a Process Model, do we need both the abstract and concrete versions? Certain processes, such as *Issue Loan*, or *Retire Member*, could deal with all kinds of member in the same way, through a generic reference to a *Member*. Other processes, such as *Create Student Member* or *Update Staff Member* may require specific variants, for example, to supply the relevant attribute values, which will be different in each case. There are different possible approaches:

- Only allow the most specific processes, dealing with concrete principal objects. This would mean that *Enrol Member* would be deleted from the model, and separate processes would handle the two concrete cases. This duplicates algorithms, loses the generality of membership, and the idea of a choice when creating different kinds of *Member*.
- Only allow the most general processes, dealing with general principal objects. This would mean that *Create Student Member*, *Create Staff Member* would be deleted from the model, and a single *Enrol Member* process would handle both cases. This delays the problem of handling specific data input cases, which must be inferred later from the kind of object.

The Process Model resolves this choice in favour of modelling the most general versions of each process. Only general processes are modelled in full, with their actions. Special processes are not elaborated further. Instead, the need to capture variations in the type(s) of data is captured in the labelling on the dataflows. The idea is that the general process will, at points where this is relevant, offer a selection of the kind(s) of data to be input by the user.

7.6 Model Transformations

The μ ML Process Model is a source or target in a number of model transformations. Some of these are to generate the Process Model from an Impact Model, or to combine a Process Model with an Object Model, or transform a Process Model forwards into something closer to detailed code.

- **ImpactToProcessModel** – is a mapping transformation that takes a μ ML Impact Model as the source, and from this constructs a μ ML Process Model. The result is a Process Model containing one Process for each Task in the Impact Model. Each Process is broken down into more basic Actions, which are determined by examining the Impacts that Tasks have on Objects in the source Impact Model. The Actions are linked by Dataflows.
- **MergeObjectWithProcess** – is a merging transformation that takes a μ ML Process Model and a μ ML Object Model as the sources, and from this constructs a revised μ ML Process Model, in which the different kinds of Datum on the Dataflows are refined, to refer to precise Attribute values, taken from the Objects. (This is a work in progress – since the task of determining full identifiers is hard, as this requires expanding object references recursively and renaming them in ways consistent with the *ObjectToDataModel* transformation).
- **ProcessToCodeModel** – is a mapping transformation that takes a μ ML Process Model as the source, and from this constructs a μ ML Code Model. This is a procedural model that is close to program code, and from which code generators are able to output code. (This is a work in progress, as the format of the Code Model is yet to be established).

These transformations are too large to display in full here.

7.7 Differences from UML

The μ ML Process Model is inspired by pre-UML Structured Analysis and Design (SA/SD) methods, which popularised Dataflow Diagrams [2, 3, 4]. These focused on the *Process View*, describing what processing was done, to what data, independently of the order of execution. UML fails to provide a *Process View* in its Activity Diagram [5, 6, 7], because the engineer has to commit first to a particular sequencing of UML Activities, which logically belongs to the *Time View* of a software system. The advantage of dataflow analysis is that it may be performed independently of when the various processes are scheduled.

Sequencing may be determined later, according to a combination of intrinsic model constraints and explicit model-transformation policies. There are three procedural interpretations of inter-process dataflow:

- pull-semantics, where each process invokes its predecessor to obtain its inputs;
- push-semantics, where each process invokes its successor to pass on its outputs;
- call-semantics, where a master process invokes each sub-process in sequence, passing inputs to, and receiving outputs from, each sub-process in turn.

By contrast, the UML Activity Diagram is specified explicitly in terms of push-semantics [7]. Not only does this rule out other possible patterns for implementing inter-Process communication, but it also makes it very difficult to specify operations naturally modelled using pull-semantics, such as data retrieval, where UML must perform several unnatural work-arounds [7]. It treats a *Datastore* as an inexhaustible buffer that always forwards all of its *Object* tokens downstream, and these have to be filtered by an *Activity* initiated by a separate control flow, which filters the desired *Object* from the store.

The μ ML Process Model is more narrowly scoped than a traditional Dataflow Diagram, which starts with a *Context Diagram* for the whole system, and then recursively breaks this down into Processes, which may be decomposed further into more Processes, to an arbitrary level. By contrast, the highest-level μ ML Process is for an atomic Task in the μ ML Task Model, which is already fine-grained. This is broken down once into a set of primitive Actions, corresponding to identified computational steps. The focus is on capturing the detailed request-response cycle of atomic Tasks, in such a way that these Actions could later be transformed into standard algorithms. With regard to the sequencing of Actions, the μ ML Process Model says nothing beyond expressing the partial order between Actions.

7.8 References

- [1] AJH Simons. ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report*, 25 January, University of Sheffield (2023).
https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan27.pdf
- [2] T DeMarco. *Structured Analysis and Systems Specification*, Prentice Hall (1978).
- [3] CP Gane and T Sarson. *Structured Systems Analysis*, Prentice Hall (1979).
- [4] E Downs, P Claire and I Coe. *Structured Systems Analysis and Design Method*¹: Application and Context, 2nd Ed., Prentice Hall (1991).
- [5] Object Management Group. About the Unified Modeling Language Specification, Version 2.5.1. December (2017). <https://www.omg.org/spec/UML/About-UML/>
- [6] G Booch, J Rumbaugh and I Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley (1999).
- [7] C Bock. UML 2 Activity Diagrams, Part 4: Object Nodes. *Journal of Object Technology*, 3 (1), (2003), 27 – 41. http://www.jot.fm/contents/issue_2004_01.html

¹ SSADM is an acronym for Structured Systems Analysis and Design Method. SSADM was developed for the Central Computer and Telecommunications Agency, then part of the UK Government.